

Universidad Politécnica de Victoria

Ingeniería en Tecnologías de la Información

Análisis Técnico de la Máquina de Estados Finitos (FSM) de la Máquina Expendedora

Materia: Fundamentos de Inteligencia Artificial

Plataforma: Arduino Mega 2560 (firmware C++, PlatformIO)

Arturo Zapata · Fabián Ramírez · Equipo

30 de septiembre de 2026

Índice

1. Introducción y Contexto del Proyecto	2
2. Punto de Entrada: main.cpp	2
2.1. Arquitectura de Super-Loop	2
2.2. Inyección de Dependencias	2
3. La Máquina de Estados: VmFsm	3
3.1. Concepto y Rol	3
3.2. Mecanismo de Transición	3
3.3. Temporización No Bloqueante	3
3.4. Seguridad Física: Sensor de Puerta	4
4. Interrelación con las Demás Clases	5
5. Cálculo de Cambio: VmChangeCalculator	5
5.1. Algoritmo Greedy Decreciente	5
5.2. Decisiones de Diseño	6
5.3. Invocación desde la FSM	6
6. Los 13 Estados de la FSM	7
6.1. Tabla Resumen	7
6.2. Descripción Detallada de Cada Estado	7
7. Diagrama UML de Actividades	11
8. Conclusiones	12

1. Introducción y Contexto del Proyecto

La máquina expendedora es un sistema embebido autónomo implementado en un **Arduino Mega 2560**. Todo el firmware corre en un único microcontrolador sin dependencias de red, corriendo sobre un *super-loop* no bloqueante.

Los periféricos que gestiona el sistema son:

- **Teclado matricial 4×4** (D22–D29) — entrada del usuario.
- **LCD 20×4 I²C** (0x27) — retroalimentación visual.
- **Motores DC** vía PCA9685 (I²C) — despacho de producto.
- **Lector RFID MFRC522** (SPI: D53, D8) — pago con tarjeta.
- **Sensor de puerta** (D3, pull-up) — seguridad física.
- **Barrera óptica de caída** (D2, pull-up) — confirmación de entrega.
- **EEPROM interna** — persistencia de inventario y caja.

El corazón del firmware es el módulo *VmFsm*, una Máquina de Estados Finitos (FSM) con 13 estados que encapsula toda la lógica de negocio: selección, cobro, despacho y cálculo de cambio.

2. Punto de Entrada: main.cpp

2.1. Arquitectura de Super-Loop

El firmware sigue el patrón *bare-metal super-loop*: `setup()` inicializa los periféricos una sola vez, y `loop()` se ejecuta continuamente sin usar interrupciones ni RTOS.

```
1 void setup() {
2     Wire.begin();           // I2C antes del LCD y PCA9685
3     display.begin();
4     motor.begin();
5     rfid.begin();
6     fsm.begin();           // carga EEPROM y entra al estado S0
7 }
8
9 void loop() {
10     char key = keypad.getKey(); // sondeo no bloqueante del teclado
11     if (key != NO_KEY)
12         fsm.handleKey(key);     // despachar tecla a la FSM
13
14     fsm.update();              // avanzar la maquina de estados
15 }
```

Listing 1: Núcleo del super-loop en main.cpp

2.2. Inyección de Dependencias

Todos los subsistemas se instancian como objetos estáticos globales y se *inyectan* al constructor de *VmFsm*:

```

1 static VmDisplay      display;
2 static VmEepromData   storage;
3 static VmMotorController motor;
4 static VmRfid         rfid;
5
6 static VmFsm fsm(storage, motor, rfid, onDisplay);

```

Listing 2: Instanciación e inyección en main.cpp

La función `onDisplay()` actúa como *callback* de presentación: la FSM nunca accede directamente al LCD; simplemente llama al puntero de función `DisplayFn` que le fue entregado en construcción.

Este diseño desacopla la lógica de negocio del hardware físico y facilita pruebas unitarias sin hardware real.

3. La Máquina de Estados: VmFsm

3.1. Concepto y Rol

`VmFsm` implementa el patrón **Mediador/Controlador**: centraliza la lógica de negocio y coordina a los demás módulos sin que éstos se conozcan entre sí.

Conceptualmente es una **Máquina de Moore**: las salidas (pantallas, activación de motor, cálculo de cambio) dependen únicamente del estado actual, y las transiciones dependen de eventos (teclas, lectura RFID, expiración de temporizadores, señales de sensores).

3.2. Mecanismo de Transición

El flujo de control sigue un patrón de dos funciones complementarias:

1. `enterState(FsmState next)` — cambia `_state` y llama al gancho `onEnter*()` correspondiente que ejecuta las acciones de entrada al nuevo estado.
2. `update()` — llamado cada ciclo desde `loop()`. Sondea temporizadores, sensores y el lector RFID para generar las transiciones temporizadas y autónomas.

Las transiciones por teclado se procesan en `handleKey()`, que traduce el carácter físico a una acción semántica mediante `VmKeypad` y la delega al procesador de teclas del estado activo.

3.3. Temporización No Bloqueante

Todos los temporizadores se implementan comparando `millis()` contra una referencia guardada, sin usar `delay()`:

```

1 // Arrancar el temporizador:
2 _motorTimer = millis();
3
4 // Verificar expiración (en update()):
5 if ((uint32_t)(millis() - _motorTimer) >= VM_TIMEOUT_MOTOR_MS) {
6     // ... acción de timeout

```

7 } |

Listing 3: Patrón de temporizador no bloqueante

El cast a `uint32_t` garantiza el manejo correcto del desbordamiento (*overflow*) del contador de 32 bits cada 49 días.

Los timeouts definidos son:

Constante	Valor	Uso
VM_TIMEOUT_INACTIVITY_MS	180 000 ms	Regreso a reposo por inactividad en S3–S6
VM_TIMEOUT_MOTOR_MS	10 000 ms	Tiempo máximo de giro del motor en S8
VM_CAROUSEL_INTERVAL_MS	2 000 ms	Rotación de subpantalla del carrusel

Cuadro 1: Temporizadores de la FSM

3.4. Seguridad Física: Sensor de Puerta

Antes de girar el motor se verifica que la puerta del cajón esté cerrada. El sensor está en D3 con nivel activo LOW. Para evitar falsos positivos por rebotes mecánicos, se implementa un filtro de debounce de 30 ms:

```

1 void VmFsm::updateDoor() {
2     bool level = (digitalRead(VM_PIN_DOOR) == VM_DOOR_CLOSED_LEVEL);
3     if (level != _doorSampled) {
4         _doorSampled = level;
5         _doorLastChangeMs = millis();
6     } else if ((uint32_t)(millis() - _doorLastChangeMs) >=
7         VM_DOOR_DEBOUNCE_MS) {
8         _doorStable = level; // estado estable confirmado
9     }
10 }
```

Listing 4: Filtro de rebote en updateDoor()

4. Interrelación con las Demás Clases

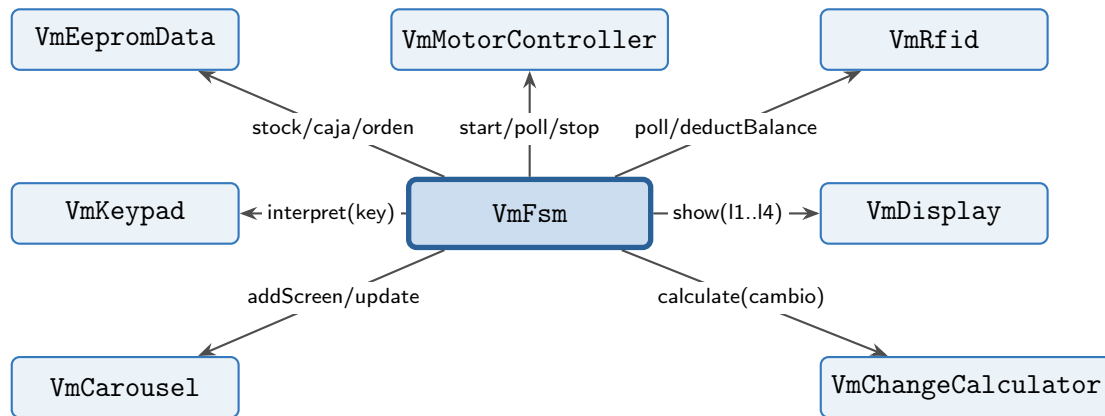


Figura 1: Dependencias de **VmFsm** con los subsistemas del firmware

La tabla siguiente resume la responsabilidad de cada módulo:

Clase	Responsabilidad
VmEepromData	Persistencia de slots (nombre, precio, stock), caja de efectivo y registro transaccional ante cortes de energía.
VmMotorController	Controla motores DC vía PCA9685; sondea la barrera óptica (D2) para confirmar la entrega.
VmRfid	Lectura de tarjetas MIFARE y escritura del nuevo saldo tras el cobro.
VmKeypad	Interpreta caracteres físicos del teclado 4×4 en acciones semánticas (KeyAction) según el contexto (KeyMode).
VmDisplay	Abstracción del LCD 20×4 I ² C; recibe 4 cadenas de texto.
VmCarousel	Rota subpantallas cada 2 s llamando al callback de display.
VmChangeCalculator	Algoritmo voraz de cálculo de cambio sobre las monedas de la caja.

Cuadro 2: Módulos del firmware y sus responsabilidades

5. Cálculo de Cambio: **VmChangeCalculator**

5.1. Algoritmo Greedy Decreciente

El cambio se calcula con un **algoritmo voraz** (*greedy*): se intentan primero las monedas de mayor denominación, reduciendo el monto restante hasta completarlo o agotar las existencias.

Las denominaciones utilizadas (en centavos de peso mexicano) son:

\$10.00 (1000¢) \$5.00 (500¢) \$2.00 (200¢) \$1.00 (100¢)

```
1  const uint32_t denominaciones[4] = { 1000u, 500u, 200u, 100u };
2
3  for (uint8_t i = 0; i < 4 && total > 0u; i++) {
4      uint32_t disponible = 0;
5      cashBox.getCoinStock(denominaciones[i], disponible);
6
7      uint32_t usar = 0;
8      while (usar < disponible && total >= denominaciones[i]) {
9          total -= denominaciones[i];
10         usar++;
11     }
12
13     if (usar > 0u) {
14         cashBox.deductCoins(denominaciones[i], usar);
15         *contadores[i] = usar;
16     }
17 }
18
19 if (total > 0u) return false; // cambio no realizable
```

Listing 5: Núcleo del algoritmo greedy en VmChangeCalculator::calculate()

5.2. Decisiones de Diseño

- **Aritmética entera en centavos:** los precios y montos se almacenan como uint32_t en centavos para evitar errores de punto flotante en el AVR de 8 bits.
- **Consulta antes del descuento:** se consulta getCoinStock() antes de descontar, garantizando que nunca se descuenta más de lo disponible.
- **Integración con EEPROM:** cada deductCoins() persiste inmediatamente en EEPROM, de modo que ante un corte de energía la caja refleja el estado real.
- **Retorno false si no hay fondos:** la FSM muestra un mensaje informativo y regresa a reposo tras 5 s sin intentar ningún pago forzado.

5.3. Invocación desde la FSM

El cálculo se inicia en el estado **S11_CALC_CAMBIO** solo si el pago fue en efectivo y si insertado > precio:

```
1  void VmFsm::onEnterCalcCambio() {
2      uint32_t cambio = _insertedCentavos - _slotInfo.priceCentavos;
3      if (cambio == 0u) {
4          enterState(FsmState::S12_PANTALLA_FIN); return;
5      }
6      bool ok = _changeCalc.calculate(_insertedCentavos,
7                                     _slotInfo.priceCentavos,
8                                     _data, _changeResult);
9      if (!ok) { /* mostrar error y esperar 5 s */ return; }
10     enterState(FsmState::S12_PANTALLA_FIN);
11 }
```

6. Los 13 Estados de la FSM

6.1. Tabla Resumen

ID	Nombre	Propósito	Transiciones de salida
S_0	S0_ARRANQUE	Inicialización, muestra mensaje de inicio.	$\rightarrow S_2$ inmediatamente.
S_1	S1_FALLA_INTERNA	Error crítico de EEPROM; sistema detenido.	Estado terminal.
S_2	S2_REPOSO	Carrusel del catálogo, espera selección.	$\rightarrow S_3$ (tecla 1–4).
S_3	S3_SEL_CANAL	Confirmar producto elegido.	$\rightarrow S_4$ (A) / S_2 (*) / timeout 180 s.
S_4	S4_SEL_PAGO	Elegir método: efectivo o RFID.	$\rightarrow S_5$ (A) / S_6 (B) / S_2 (*).
S_5	S5_ESP_EFECTIVO	Acumular monedas hasta cubrir el precio.	$\rightarrow S_7$ (monto precio) / S_2 (B) / timeout.
S_6	S6_ESP_RFID	Sondear RFID y cobrar saldo de tarjeta.	$\rightarrow S_7$ (OK) / S_2 (error/timeout/B).
S_7	S7_RESERVADA	Verificar stock y arrancar motor.	$\rightarrow S_8$ (OK) / S_{10} (precondición fallida) / S_2 (sin stock, 1.5 s).
S_8	S8_DISPENSANDO	Motor activo; esperar barrera óptica.	$\rightarrow S_9$ (barrera OK) / S_{10} (timeout 10 s).
S_9	S9_CONFIRMADA	Entrega exitosa; esperar 1 s.	$\rightarrow S_{11}$ (efectivo) / S_{12} (RFID).
S_{10}	S10_FALLA_DISP	Rollback de inventario; mensaje error 3 s.	$\rightarrow S_2$.
S_{11}	S11_CALC_CAMBIO	Calcular y deducir monedas de cambio.	$\rightarrow S_{12}$ (éxito) / S_2 (sin fondos, 5 s).
S_{12}	S12_PANTALLA_FIN	Carrusel final con resumen y cambio.	$\rightarrow S_2$ (tras 2 ciclos de carrusel).

Cuadro 3: Los 13 estados de la FSM de la máquina expendedora

6.2. Descripción Detallada de Cada Estado

S_0 — **Arranque.** Se muestra un mensaje de bienvenida mientras `begin()` carga la EEPROM. Si los datos son válidos, la transición a S_2 ocurre en el primer ciclo de `update()`.

S_1 — **Falla Interna.** Estado terminal alcanzado si `VmEepromData::begin()` devuelve `false`. El sistema muestra un error permanente y no acepta ninguna entrada. Requiere reinicio físico.

S_2 — **Reposo.** Estado de espera de largo plazo. El carrusel rota mostrando dos productos por subpantalla cada 2 s. Es el destino de retorno de todos los flujos de error, cancelación y éxito.

S_3 — **Selección de Canal.** Se consulta `VmEepromData::getSlot()` para verificar que el slot está habilitado. Si no, regresa a S_2 inmediatamente. Muestra nombre del producto y precio. La tecla A confirma.

S_4 — **Selección de Pago.** El usuario elige la pasarela de cobro. Si selecciona RFID pero el lector no está disponible (`rfid.isAvailable()` devuelve `false`), se muestra aviso y se permanece en la pantalla de selección.

S_5 — **Espera de Efectivo.** Las teclas 1–9 representan monedas de \$1 a \$500. Cada pulsación llama a `addCoins()` en EEPROM y actualiza el acumulado `_insertedCentavos`. Cuando el acumulado cubre el precio, se transita automáticamente a S_7 .

S_6 — **Espera de RFID.** `VmRfid::poll()` se llama en cada ciclo sin bloquear. Si la lectura es exitosa, se verifica el saldo. Si es suficiente, se llama a `deductBalance()` y se transita a S_7 . Si el saldo es insuficiente o hay error de lectura, se muestra un mensaje y se regresa a S_2 .

S_7 — **Reservada.** Punto de no retorno transaccional. Se ejecuta en orden: (1) `reserveStock()` — decrementa inventario; (2) `beginOrder()` — persiste en EEPROM que hay una orden en curso; (3) Se verifica `_doorStable` (puerta cerrada); (4) `motor.start()` — verifica que la barrera esté libre y enciende el motor. Si cualquier condición falla, se transita a S_{10} .

S_8 — **Dispensando.** En cada ciclo de `update()`, se consulta `motor.isBusy()`. Cuando la barrera óptica detecta la caída del producto, `VmMotorController` detiene el motor y pone el resultado en `consumeResult()`. Un timeout de 10 s protege contra atascos.

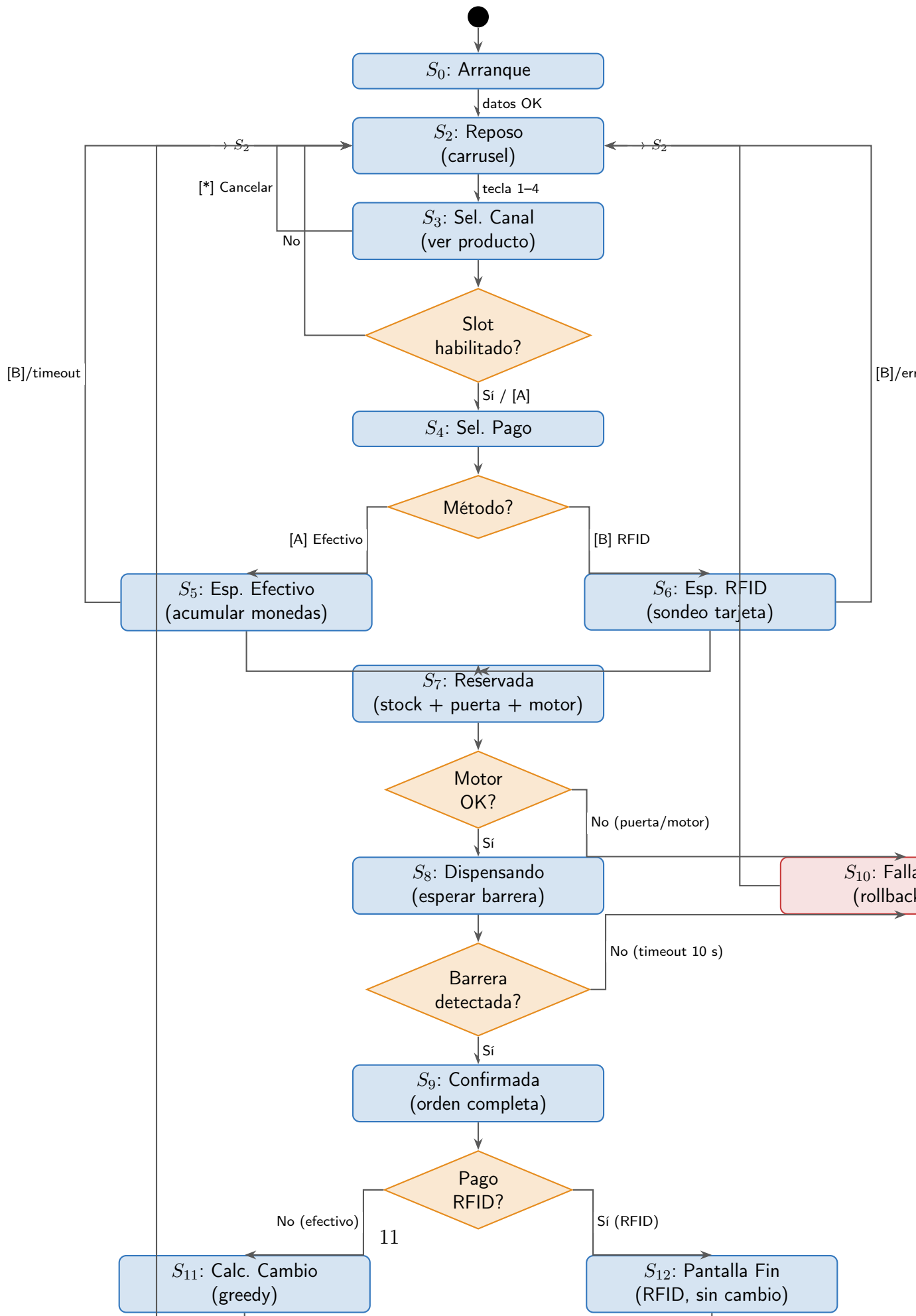
S_9 — **Confirmada.** Se llama a `completeOrder()` para cerrar el registro transaccional en EEPROM. La pantalla de éxito se muestra 1 s antes de avanzar a S_{11} (efectivo) o S_{12} (RFID, sin cambio).

S_{10} — **Falla de Despacho.** Si `_stockReserved` es `true`, se deshace la operación: `completeOrder()` con el resultado de falla y `releaseStock()` devuelven el inventario. El mensaje de error dura 3 s y retorna a S_2 .

S_{11} — **Cálculo de Cambio.** Invoca `VmChangeCalculator::calculate()` que aplica el algoritmo greedy y descuenta las monedas usadas de la EEPROM. Si no hay fondos suficientes, se muestra aviso y se espera 5 s antes de volver a S_2 .

S_{12} — **Pantalla Final.** El carrusel muestra dos subpantallas: la primera confirma la compra, la segunda detalla las monedas de cambio (o el cobro RFID). Tras dos intervalos de 2 s más 1 s de margen, retorna a S_2 .

7. Diagrama UML de Actividades



8. Conclusiones

El diseño de la FSM cumple los objetivos de un sistema embebido robusto:

1. **Legibilidad lineal:** los 13 estados están numerados secuencialmente S_0 – S_{12} y el código fuente sigue el mismo orden de arriba a abajo, facilitando la lectura y el mantenimiento.
2. **Sin sobreingeniería:** cada clase tiene una responsabilidad única y bien delimitada. La FSM no accede directamente al hardware; toda comunicación es a través de las interfaces de los subsistemas.
3. **Robustez ante cortes de energía:** el estado transaccional de cada orden se persiste en EEPROM en S_7 (`beginOrder`) y se cierra en S_9/S_{10} (`completeOrder`), garantizando consistencia aunque la alimentación se corte durante el despacho.
4. **Temporización no bloqueante:** todos los delays del sistema se implementan con `millis()`, permitiendo que el super-loop permanezca siempre receptivo a eventos externos.
5. **Trazabilidad de fallos:** el estado S_{10} y la variable `_pendingResult` permiten distinguir tres causas de falla de despacho y presentar mensajes diferenciados al usuario.